

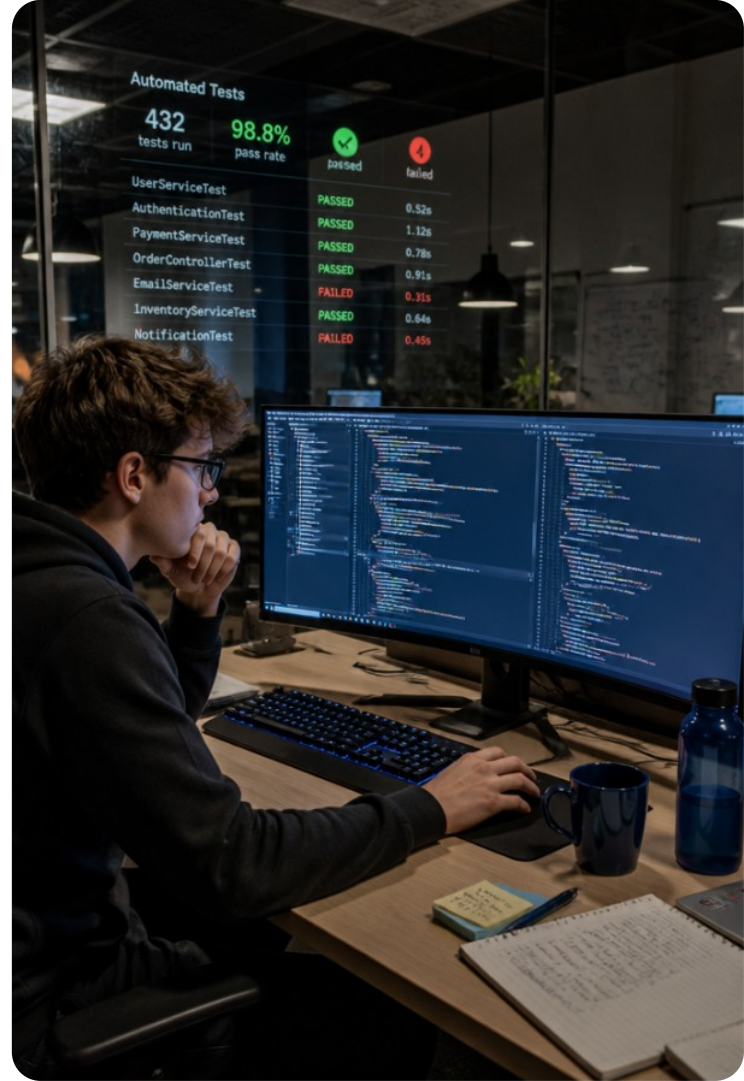


Testes Unitários no Frontend com Vitest

Automação rápida, isolamento e qualidade técnica de software moderno.

O Desafio do Código Invisível

Se você altera uma única função no frontend hoje, como ter certeza absoluta de que o checkout de pagamentos não parou de funcionar sem clicar em toda a interface manualmente?



Objetivos de Aprendizagem

Metas da Aula

Nesta aula prática, você dominará pilares de testes:

Configurar **Vitest** em projetos Node.js/npm.

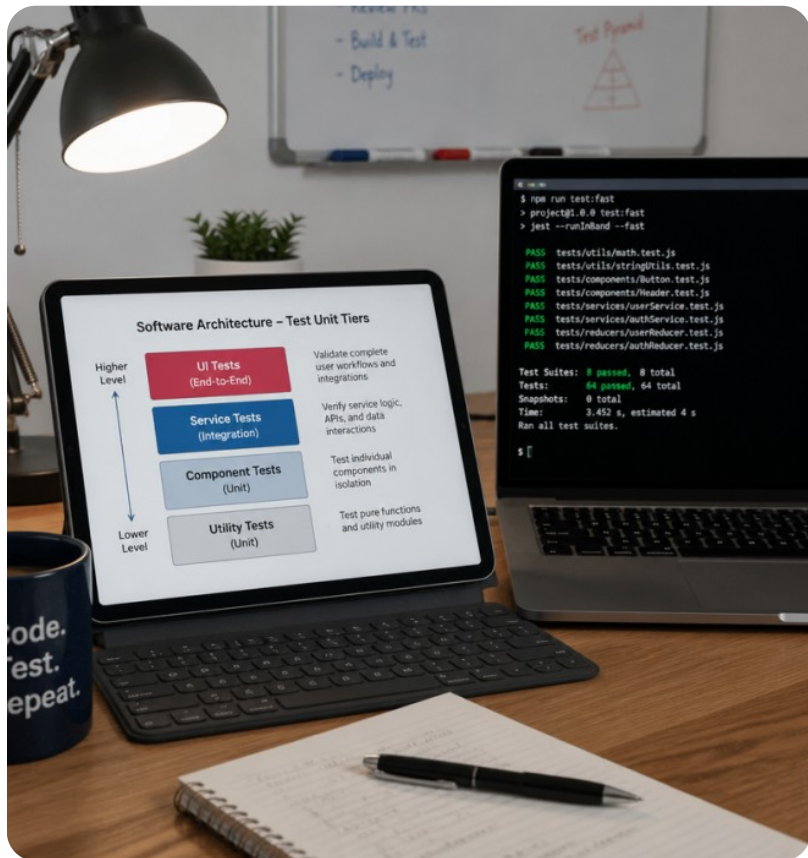
Estruturar testes com o **padrão AAA**.

Implementar **matchers**, **mocks** e **spies**.

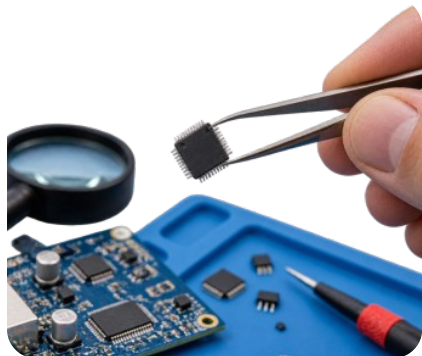


Lembre-se

Testes unitários são a base da pirâmide: rápidos e essenciais para refatorações seguras.



Vocabulário Essencial



Unitário: Teste focado na menor parte testável de código de forma isolada.



Matcher: Método de asserção que valida se o resultado obtido equivale ao esperado.



Isolamento: Técnica que separa a unidade sob teste de dependências como rede e banco.



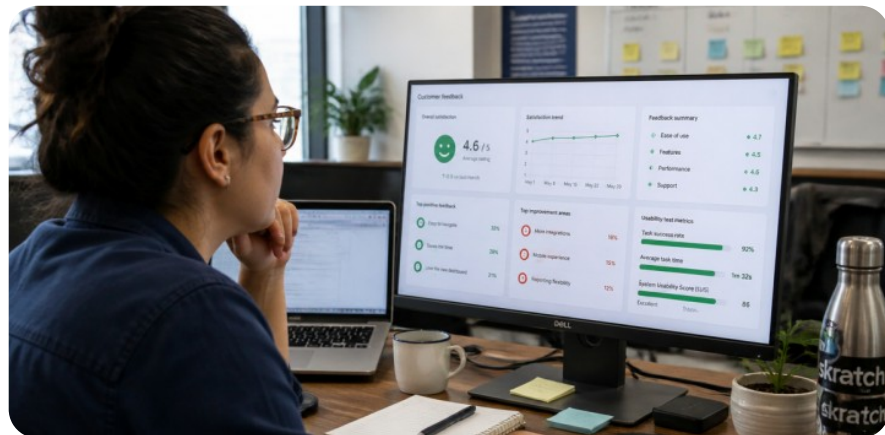
Simulação: Substituição programada de comportamentos reais (APIs, timers) por réplicas.

Recapitulação: Verificação e Validação



Verificação

Responde à pergunta: **O produto está sendo construído corretamente?** Foca na conformidade com especificações técnicas e regras de arquitetura.



Validação

Responde à pergunta: **Estamos construindo o produto certo?** Avalia se o sistema atende às necessidades reais do usuário final e regras de negócio.

Recapitulação: Plano e Casos BDD

PLANEJAMENTO

Formalizamos o plano de testes e convertemos requisitos em BDD:

Given: Estado inicial do sistema.

When: Ação do usuário ou evento.

Then: Resultado esperado e mensurável.



Ponto-chave

Casos BDD mapeiam diretamente para `describe()` e `it()` do Vitest.



Quiz: Verificação e Planejamento



Pergunta 1:

Qual é a principal distinção entre Verificação e Validação no ciclo de testes?

Pergunta 2:

No padrão BDD (Behavior Driven Development), o que define a cláusula 'When'?

Pergunta 3:

Por que a base da Pirâmide de Testes é composta primariamente por testes unitários?

Quiz: Verificação e Planejamento



Resposta 1:

Verificação checa se o software segue as especificações técnicas, enquanto validação checa se ele resolve o problema real do usuário.

Resposta 2:

A ação ou evento específico disparado que altera o estado do sistema.

Resposta 3:

Porque são os testes mais rápidos de executar, mais fáceis de isolar e com menor custo de manutenção.

Apresentando o Vitest

FERRAMENTA

O **Vitest** é um test runner moderno para Vite. Usa o mesmo pipeline de transformação, superando a lentidão de ferramentas legadas.

Velocidade: Compilação sob demanda com ES Modules.

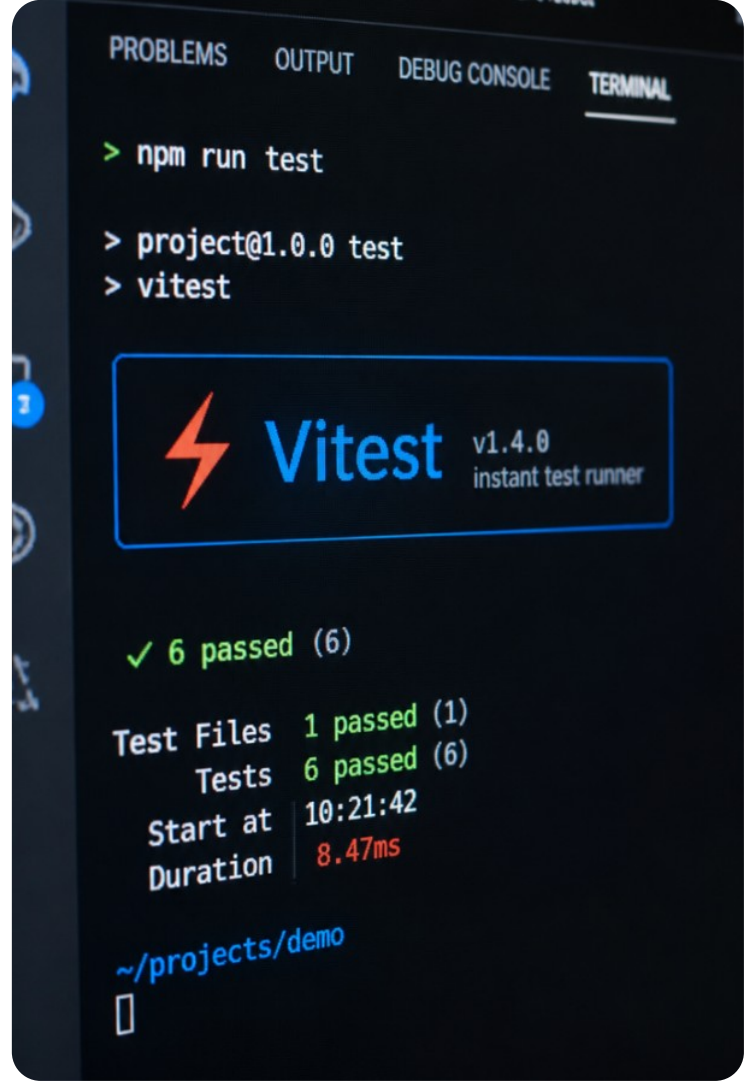
Compatibilidade: API idêntica ao Jest.

Watch Mode: Re-execução em milissegundos.



Curiosidade

Executa testes TypeScript nativamente sem compilação intermediária.



Arquitetura e Velocidade do Vitest

Veja como o Vitest transforma o fluxo de trabalho do desenvolvedor ao unificar configuração e compilação em um ecossistema veloz.



A execução multithread paralela reduz suites de dezenas de segundos para milissegundos.

Configuração do Ambiente

A inicialização de um projeto moderno de testes requer apenas quatro etapas padronizadas via terminal:



Estrutura do package.json

CONFIGURAÇÃO

O arquivo `package.json` gerencia as dependências de desenvolvimento e os comandos executáveis do seu projeto.

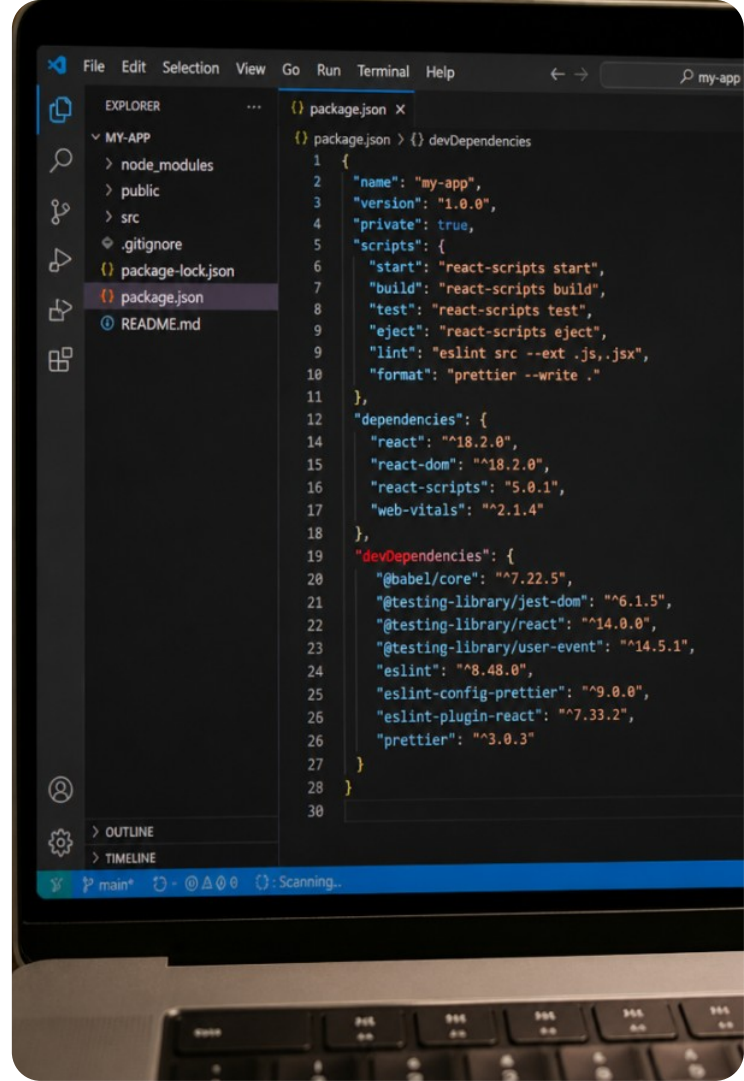
O atributo `type` habilita `import/export` nativo do ES6.

- O script de teste permite que ferramentas de Integração Contínua (CI) executem sua validação com comando padronizado.



Exemplo

No `package.json`: defina `test` para rodar os testes uma única vez na esteira de CI.



Convenções de Nomenclatura

ORGANIZAÇÃO

O Vitest detecta arquivos de teste automaticamente via convenções:

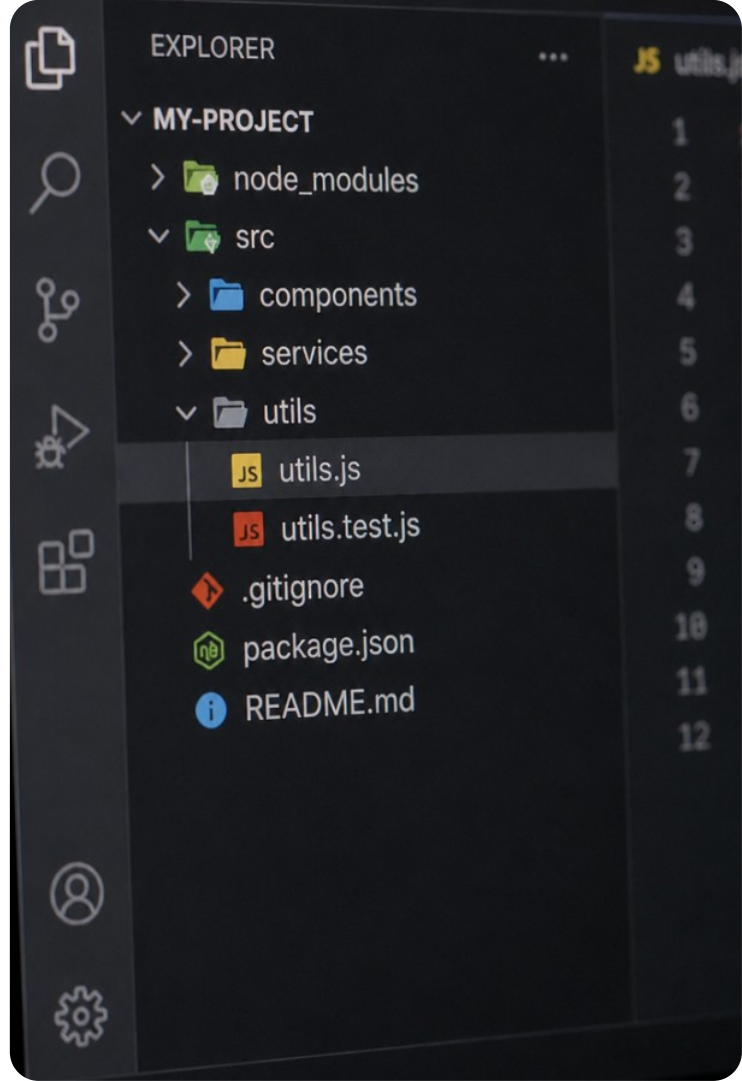
- Padrão para testes unitários.
- Usado para especificações de comportamento.

Pasta : Para agrupar suites complexas.



Ponto-chave

Mantenha ao lado de para facilitar a manutenção.



Sintaxe Base: describe, it e test



describe(titulo, fn)

Agrupar um conjunto de testes correlacionados. Cria uma suite lógica que contextualiza o módulo ou componente avaliado.



it(descricao, fn) ou test()

Declara um caso de teste individual. O nome estimula a leitura semântica: *'it should calculate total price with discount'*.

A Função expect() e Asserções

SINTAXE

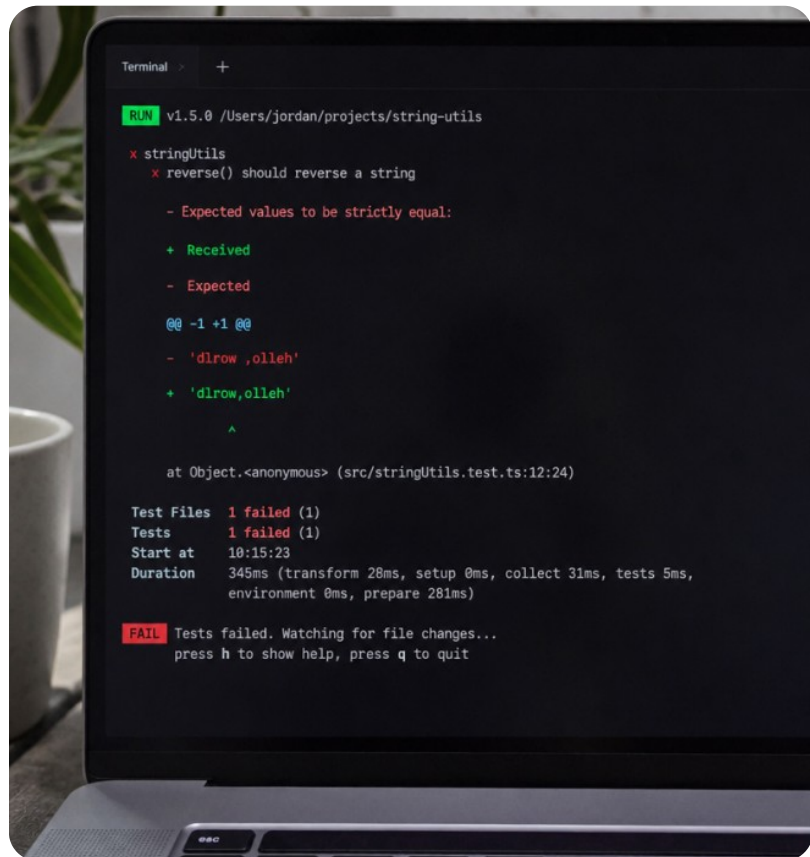
A asserção é o veredito do teste. encapsula o dado e expõe matchers.

- Se a condição for atendida, o teste passa (verde).
- Se divergir, o Vitest interrompe o bloco e exibe o diff exato no terminal.



Lembre-se

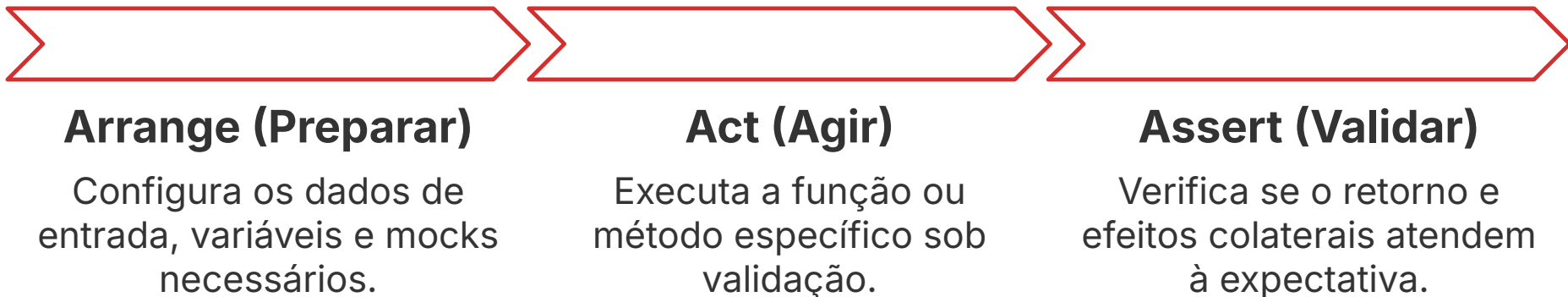
Nunca crie um teste sem expect().
Testes sem asserção geram falsa segurança.



```
Terminal > +  
  
RUN v1.5.0 /Users/jordan/projects/string-utils  
  
x stringUtils  
  x reverse() should reverse a string  
  
    - Expected values to be strictly equal:  
  
    + Received  
    - Expected  
  
    @@ -1 +1 @@  
  
    - 'dlrow ,olleh'  
    + 'dlrow,olleh'  
      ^  
  
    at Object.<anonymous> (src/stringUtils.test.ts:12:24)  
  
Test Files  1 failed (1)  
Tests      1 failed (1)  
Start at   10:15:23  
Duration   345ms (transform 28ms, setup 0ms, collect 31ms, tests 5ms,  
            environment 0ms, prepare 281ms)  
  
FAIL Tests failed. Watching for file changes...  
       press h to show help, press q to quit
```

O Padrão AAA: Estrutura Universal

O padrão Arrange-Act-Assert organiza o raciocínio em três etapas lineares e previsíveis:



Dissecando o Padrão AAA em Código

Exemplo de Teste de Cálculo de Desconto

Manter comentários ou separação visual explícita entre as três fases torna o código autoexplicativo para qualquer colega de equipe.



Ponto-chave

Mantenha a fase 'Act' em uma única linha sempre que possível para garantir foco na ação testada.

Ordem dos Fatos: Ciclo AAA

Ordene as etapas canônicas de execução de um teste unitário segundo o padrão AAA:

**Comparar o valor retornado contra o matcher
esperado com expect (Assert)**

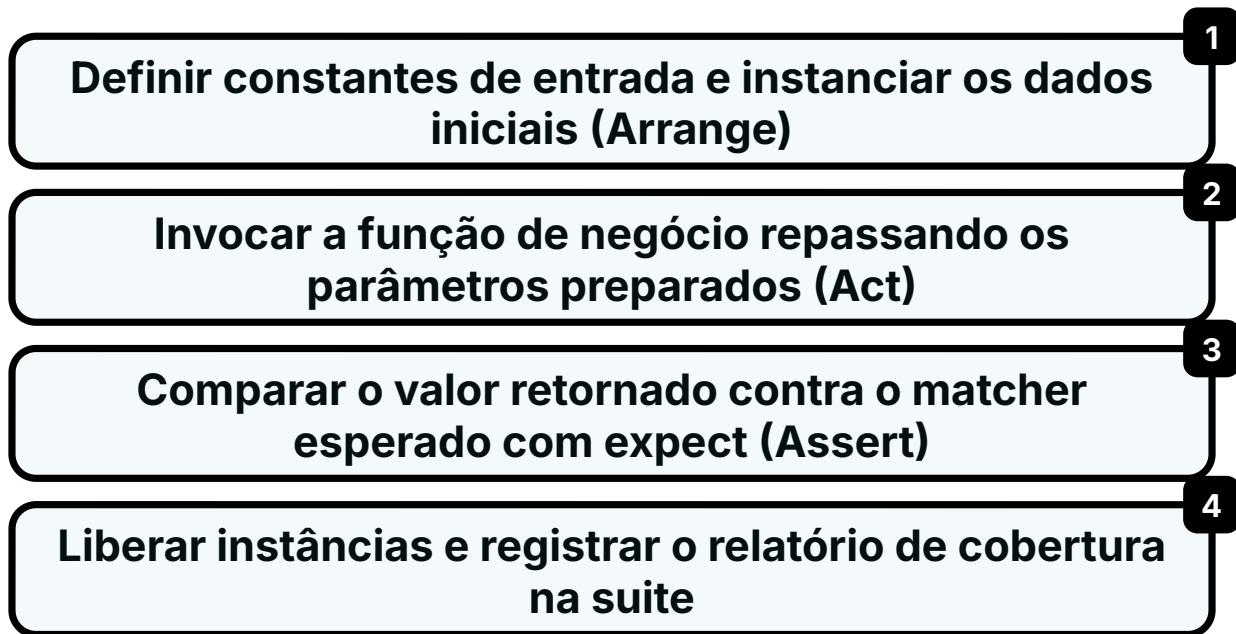
**Liberar instâncias e registrar o relatório de cobertura
na suite**

**Definir constantes de entrada e instanciar os dados
iniciais (Arrange)**

**Invocar a função de negócio repassando os
parâmetros preparados (Act)**

Ordem dos Fatos: Ciclo AAA

Ordene as etapas canônicas de execução de um teste unitário segundo o padrão AAA.



Matchers de Igualdade: toBe vs toEqual



toBe()

Verifica **igualdade estrita de referência ou valor primitivo** (). Ideal para comparar números, strings e booleanos.



toEqual()

Realiza **comparação profunda de estruturas** (deep equality). Essencial para objetos, arrays e coleções complexas.

Armadilha Clássica: Referência de Objetos

BOAS PRÁTICAS

Objetos literais em JS nunca compartilham a mesma referência, mesmo com chaves e valores idênticos.

→ **FALHA** (endereços distintos).

→ **SUCESSO** (conteúdo idêntico).



Atenção

Usar `toBe()` em arrays/objetos é o erro mais comum de iniciantes.

The slide illustrates the difference between reference and value equality for JavaScript objects. It shows two identical JSON objects, A and B, each with properties `id`, `name`, and `isActive`.

JSON Object A

```
{
  "id": 1,
  "name": "Alex",
  "isActive": true
}
```

Memory Addresses (Example)

Key	Value	Memory Address
id	1	0x7ffdf1a2b0e8
name	Alex	0x7ffdf1a2b100
isActive	true	0x7ffdf1a2b108

Direct Pointer Comparison

Object A	Object B
0x7ffdf1a2b0e0	0x7ffdf1a2c1f0

✗ Not Equal
(Different memory addresses)

JSON Object B

```
{
  "id": 1,
  "name": "Alex",
  "isActive": true
}
```

Memory Addresses (Example)

Key	Value	Memory Address
id	1	0x7ffdf1a2c1f8
name	Alex	0x7ffdf1a2c210
isActive	true	0x7ffdf1a2c218

Deep Value Comparison

All keys and values are the same

✗ Objects are not equal by reference
(But equal by value)

✗ Even though the JSON content is identical, the memory addresses (pointers) are different. Therefore, the two objects are **not equal** when compared by reference.

Matchers de Estado e Coleções



toBeNull() -
Verifica se o valor
é , ideal para
buscas sem
resultados.



toBeDefined() -
Confirma que a
variável foi
inicializada e não é
.



toContain() -
Checa se um item
existe em um array
ou se um texto
contém substring.



toBeGreaterThan()
- Valida
comparações
numéricas, útil
para limites
mínimos.

Checagem de Matchers

Qual asserção deve ser usada para conferir se a lista de permissões ['ADMIN', 'EDITOR'] contém a role 'ADMIN'?

1. `expect(permissoes).toContain('ADMIN')`
2. `expect(permissoes).toEqual('ADMIN')`
3. `expect(permissoes).toBeDefined('ADMIN')`
4. `expect(permissoes).toBe('ADMIN')`

Checagem de Matchers



Qual asserção deve ser usada para conferir se a lista de permissões ['ADMIN', 'EDITOR'] contém a role 'ADMIN'?

1. `expect(permissoes).toContain('ADMIN')`

2. `expect(permissoes).toEqual('ADMIN')`

3. `expect(permissoes).toBeDefined('ADMIN')`

4. `expect(permissoes).toBe('ADMIN')`

O Conceito de Mock e Espião

ISOLAMENTO

Funções reais dependem de fatores externos: rede, relógio ou alertas.

- APIs externas tornam testes lentos e instáveis.
- Mocks e Spies interceptam chamadas, devolvendo respostas controladas e imediatas.

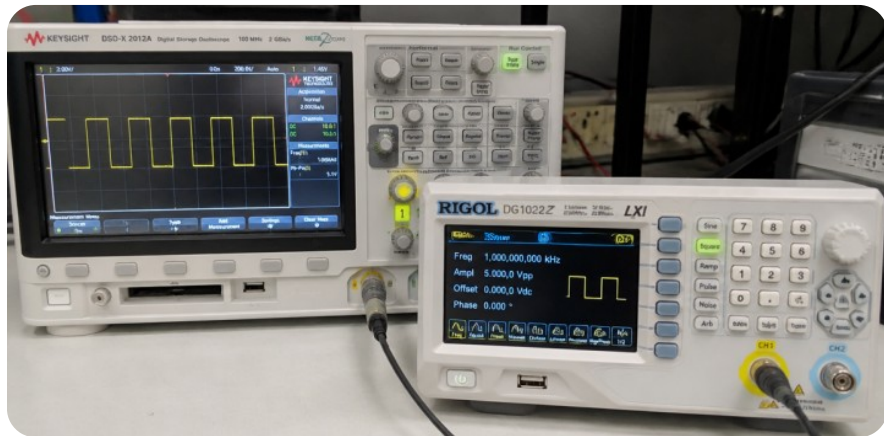


Ponto-chave

Substitua chamadas assíncronas por simulações determinísticas em testes unitários.



Simulando Funções com vi.fn()



O que faz vi.fn()?

Cria uma função simulada que memoriza quantas vezes foi chamada, quais argumentos recebeu e qual valor ela deve devolver sob demanda.



Validação com toHaveBeenCalled

Podemos validar: ou verificar o argumento exato com .

Monitorando Métodos com vi.spyOn()

ISOLAMENTO

Diferente de `vi.fn()` que gera uma função do zero, envolve um método já existente em um objeto real.

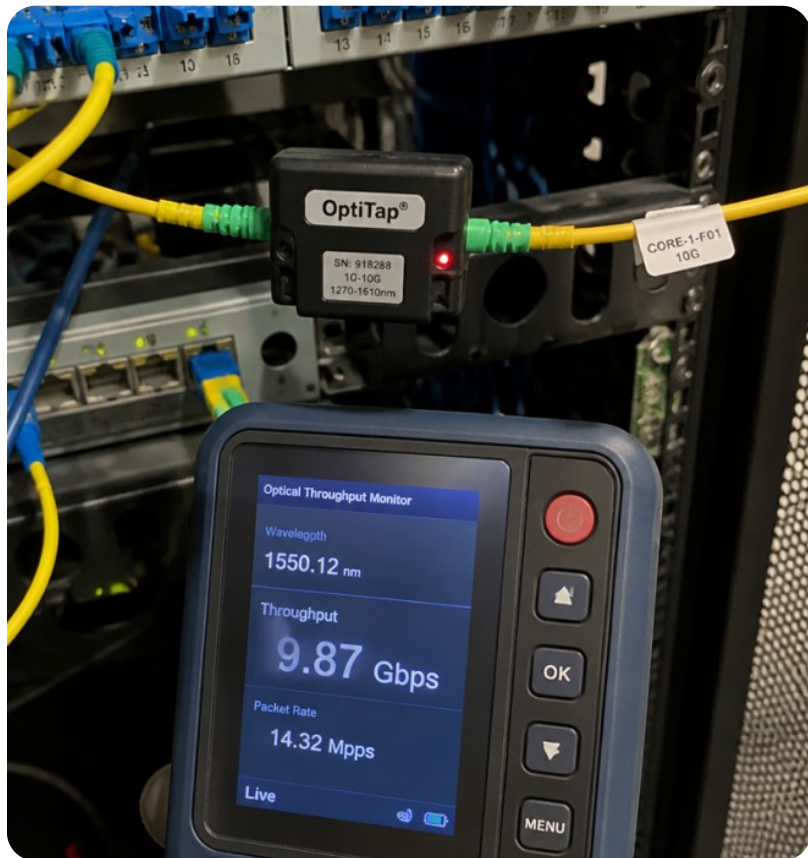
Permite observar se o método `ou` foi acionado pela sua função.

Permite sobrescrever temporariamente o comportamento real usando `vi.restoreAllMocks()` ou `vi.restoreMock()`.



Lembre-se

Sempre chame `vi.restoreAllMocks()` após cada teste para evitar contaminação entre suites.



Vitest na Prática: Demonstração em Vídeo

Configuração do Vitest e uso de matchers essenciais.



Prática 1: Funções Utilitárias

Especificação das 5 Funções para Teste

Vamos implementar e exercitar suites de testes unitários para o módulo :

- 1.: Retorna booleano conferindo presença de @ e domínio válido.
- 1.: Retorna valor fixo se menor que 10km, ou taxa por km excedente.
- : Formata centavos em string pt-BR padronizada ().
- 1.: Devolve lista purificada com itens únicos usando Set.
- 1.: Extrai as duas primeiras letras capitais dos nomes principais.



Ponto-chave

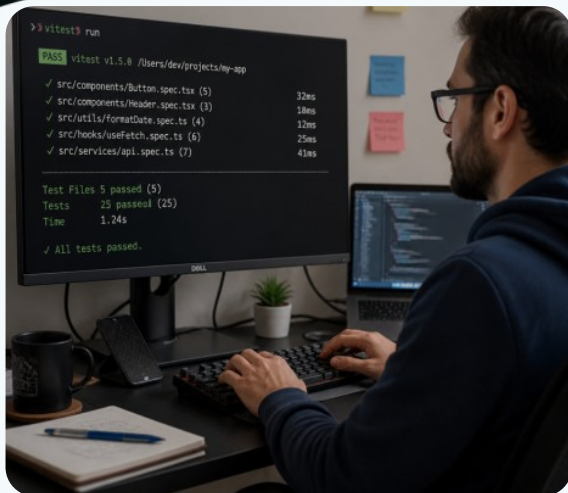
Cada teste deve explorar o caminho feliz e ao menos um caso de borda (ex.: valores nulos ou strings vazias).

Implementação da Prática 1

Código da Suite (testes 1 a 3)

Código da Suite (testes 4 e 5)

Atividade Prática 1: Execução



Abra seu terminal no diretório do projeto e implemente o arquivo com os 5 cenários listados no slide anterior. Em seguida execute e garanta que todos os 5 testes passem com 100% de sucesso. Identifique no console qual matcher foi responsável por validar cada um dos 5 tipos de dados (primitivo, array, string e nulo).

Prática 2: Mocks em Funções Assíncronas

Cenário: Buscar Usuário via API Externa

Considere a função que faz uma requisição HTTP assíncrona. Jamais devemos disparar conexões de rede em testes unitários. Veja a montagem do mock com :

1. Preparação (Arrange)

2. Ação e Asserção (Act & Assert)

Atividade Prática 2: Mock de API

```
authService.js
1  const authenticateUser = async (username, password) => {
2    try {
3      const user = await getUserByUsername(username);
4      if (!user) {
5        throw new Error('User not found');
6      }
7
8      const isValid = await verifyPassword(password, user.passwordHash);
9      if (!isValid) {
10       throw new Error('Invalid password');
11     }
12
13     const token = await generateToken(user.id);
14     return { success: true, token, userId: user.id };
15   } catch (error) {
16     console.error('Authentication error:', error.message);
17     return { success: false, error: error.message };
18   }
19 };
20
21 const getUserByUsername = async (username) => {
22   // Mock: simulate async DB lookup
23   return username === 'testuser'
24     ? { id: 1, username: 'testuser', passwordHash: 'hashed_pw_123' }
25     : null;
26 };
27
28 const verifyPassword = async (password, hash) => {
29   // Mock: simulate password check
30   return password === 'Password123!' && hash === 'hashed_pw_123';
31 };
32
33 const generateToken = async (userId) => {
34   // Mock: simulate token generation
35   return `mock_jwt_token_for_user_${userId}`;
36 };
37
```

Implemente uma suite de testes que valida uma função assíncrona de autenticação . Utilizando , teste o comportamento de erro do sistema quando o servidor recusa o login. Assegure-se de verificar se a função lida adequadamente com o erro usando .

Armadilhas Comuns em Testes Unitários

BOAS PRÁTICAS

Evite testes frágeis ou intermitentes (*flaky*):

Ordem: Testes devem ser independentes.

Estado: Limpe variáveis globais entre execuções.

Foco: Teste comportamento, não detalhes internos.



Atenção

Testes frágeis perdem a confiança e inutilizam a automação.

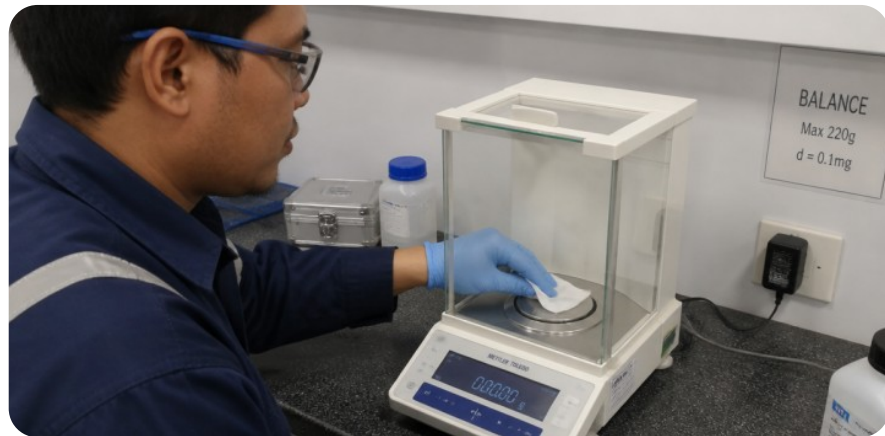


Ciclo de Vida: beforeEach e afterEach



beforeEach()

Executado antes de cada teste individual da suite. É o local perfeito para redefinir objetos, recriar instâncias e inicializar bancos em memória.



afterEach()

Executado logo após o término de cada teste. Utilizado para chamar e remover timers pendentes que possam poluir o ambiente.

Conceito: Isolamento de Testes

Um teste unitário ideal deve depender da execução bem-sucedida do teste anterior para reaproveitar os dados criados no banco.



VERDADE
IRO



FALSO



Prepare-se para explicar o seu raciocínio.

Conceito: Isolamento de Testes



Um teste unitário ideal deve depender da execução bem-sucedida do teste anterior para reaproveitar os dados criados no banco.



Por que é isso?

Cada teste unitário precisa ser 100% independente e isolado, garantindo execução em qualquer ordem ou em paralelo.

Consolidação de Conceitos Vitest

Relacione as palavras com as definições

vi.spyOn

Envolve e observa um método já existente em um objeto real.

vi.fn

Matcher para comparação de valores primitivos e igualdade estrita.

toBe

Cria uma função simulada para rastrear chamadas e retornos programados.

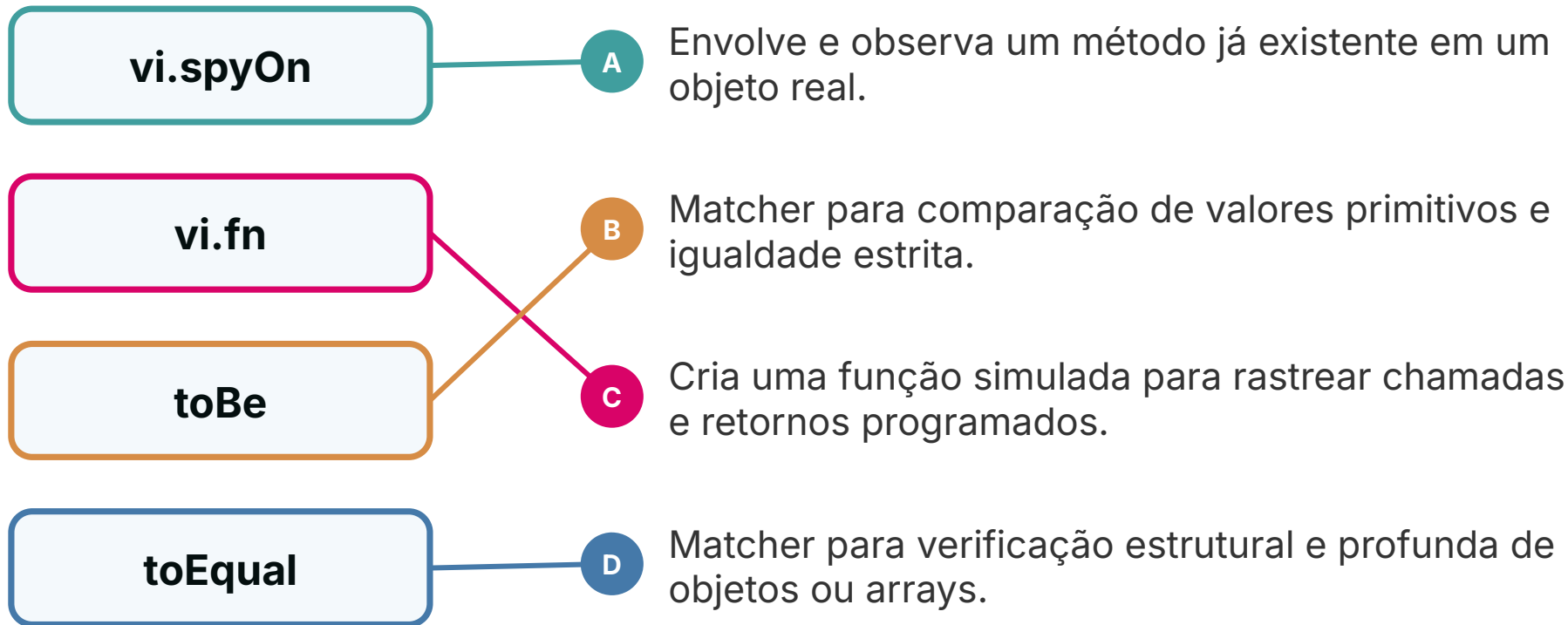
toEqual

Matcher para verificação estrutural e profunda de objetos ou arrays.

Consolidação de Conceitos Vitest



Relacione as palavras com as definições



Debate: Quanto Devemos Mockar?



Se usarmos mocks em todas as dependências, garantimos testes rápidos. Porém, qual o perigo de simularmos código demais em nossa suite de frontend?

Debate: Quanto Devemos Mockar?



Você poderia ter dito...

O risco é criar falsos positivos: o teste unitário passa com sucesso usando mocks ideais, mas em produção o contrato real da API mudou e a interface quebra.

Por isso, testes unitários com mocks devem ser equilibrados com testes de integração reais (Bloco 04 da nossa ementa).